

FCA.py 算法说明文档

版本: 基于当前 FCA.py 实现 (cross-sector hue shift + hue range taper)

1. 算法开发背景

本算法用于图像色相编辑场景。核心需求是：在调整色相(Hue)时，尽量保持亮度(Value)和饱和度(Saturation)稳定，并支持只在指定色相带内进行局部编辑。

相比“直接在 RGB 空间旋转色度”方法，本实现采用分段 HSV 语义重建，目的是让颜色变化更可控、可解释。

2. 算法目标

- 支持跨扇区色相位移: $H \rightarrow H + \Delta H$ 。
- 在重建中保持 V 与 Δ 不变，从而保持 $S = \Delta / V$ 。
- 支持色相范围掩码与边缘平滑(taper)，减轻范围边界突变。

3. 详细实现与数学公式 (学术化表述)

本章采用与 FCA_fixedpoint.py 对齐的变量体系，并给出编号公式、结论与误差项。

3.1 预备定义与有效域

设输入像素为 $x=(R,G,B)$, $R,G,B \in [0,1]$ 。定义如下：

$$V(x)=\max(R,G,B), \quad m(x)=\min(R,G,B), \quad \Delta(x)=V(x)-m(x) \quad (3-1)$$

$$S(x)=0 \quad (V=0), \quad \text{否则} \quad S(x)=\Delta(x)/V(x) \quad (3-2)$$

当 $\Delta=0$ 时 Hue 无定义，此类像素记为非活动像素并保持原值。

3.2 扇区化 Hue 表示

活动像素按 (max 通道, min 通道) 划分为 6 个扇区 $s \in \{0,1,2,3,4,5\}$ 。

在每个扇区定义线性参数 $t \in [0,1]$, 例如:

$$t=(G-B)/\Delta (s=0), t=(G-R)/\Delta (s=1), t=(B-R)/\Delta (s=2) \quad (3-3)$$

$$t=(B-G)/\Delta (s=3), t=(R-G)/\Delta (s=4), t=(R-B)/\Delta (s=5) \quad (3-4)$$

Hue 可写为:

$$H = 60*(s+t), \quad H \in [0,360) \quad (3-5)$$

对应 fixed-point 形式为 $\text{hue_hu} = s*ONE + t_fp$, 与 `FCA_fixedpoint.py` 一致。

3.3 色相范围与回绕判定

给定区间 $[hmin, hmax]$, 活动掩码 A 定义为:

$$A = \{H | hmin \leq H \leq hmax\}, \text{ 若 } hmin \leq hmax \quad (3-6)$$

$$A = \{H | H \geq hmin \text{ 或 } H \leq hmax\}, \text{ 若 } hmin > hmax \quad (3-7)$$

式(3-7)处理跨 0° 区间 (如 $[300^\circ, 30^\circ]$) 。

3.4 taper 平滑与有效位移

记 $d(H)$ 为 H 到区间边界的最小角距离, 定义:

$$\text{scale}(H) = \text{clip}(d(H)/\text{taper_width}, 0, 1) \quad (3-8)$$

$$\text{eff_dH}(H) = \text{deltaH} * \text{scale}(H) \quad (3-9)$$

当 $\text{taper_width}=0$ 时, 退化为 $\text{eff_dH}=\text{deltaH}$ 。

3.5 Hue 更新与新扇区定位

更新后的色相为:

$$H_{\text{new}} = (H + \text{eff_dH}) \bmod 360 \quad (3-10)$$

$$s_{\text{new}} = \text{floor}(H_{\text{new}}/60), \quad t_{\text{new}} = H_{\text{new}}/60 - s_{\text{new}} \quad (3-11)$$

fixed-point 对应关系: $s_{\text{new}} = \text{hue_new_hu} \gg \text{frac_bits}$, $t_{\text{new}} = \text{hue_new_hu} \& (ONE-1)$ 。

3.6 通道重建

在 s_{new} 扇区中按“max/min 固定 + 中间通道插值”重建:

$$c_max=V, \quad c_min=m=V-Delta \quad (3-12)$$

$$c_mid=c_min+Delta*t_new \text{ 或 } c_mid=c_max-Delta*t_new \quad (3-13)$$

具体采用哪一式由扇区定义决定，且与 FCA_fixedpoint.py 的 ns0..ns5 规则一一对应。

3.7 定理与结论

定理 1 (亮度保持) : 对所有活动像素，重建后 $V_{out} = V_{in}$ 。

证明要点: 式(3-12)直接将最大通道赋值为 V ，且其它通道不超过该值。

定理 2 (饱和度保持) : 对所有活动像素， $S_{out} = S_{in}$ 。

证明要点: 由式(3-12)得 $Delta_out = c_max - c_min = V - (V - Delta) = Delta$ ；再代入式(3-2)即得。

推论: 本算法的主要变化仅体现在 Hue 维度， V 与 S 在数学上保持不变。

3.8 误差项说明

理论上式(3-1)至式(3-13)在实数域可精确满足约束；工程实现误差来自以下三类:

- 1) 浮点舍入误差: 由除法和乘法产生，量级通常在 $1e-15$ 至 $1e-12$ 。
- 2) 量化误差: 保存到 uint8 时引入 ± 0.5 LSB 误差。
- 3) 定点近似误差: fixed-point 中由 frac_bits 限定，单次乘法截断误差上界约为 $2^{-(frac_bits)}$ 。

本文档采用的验证指标为:

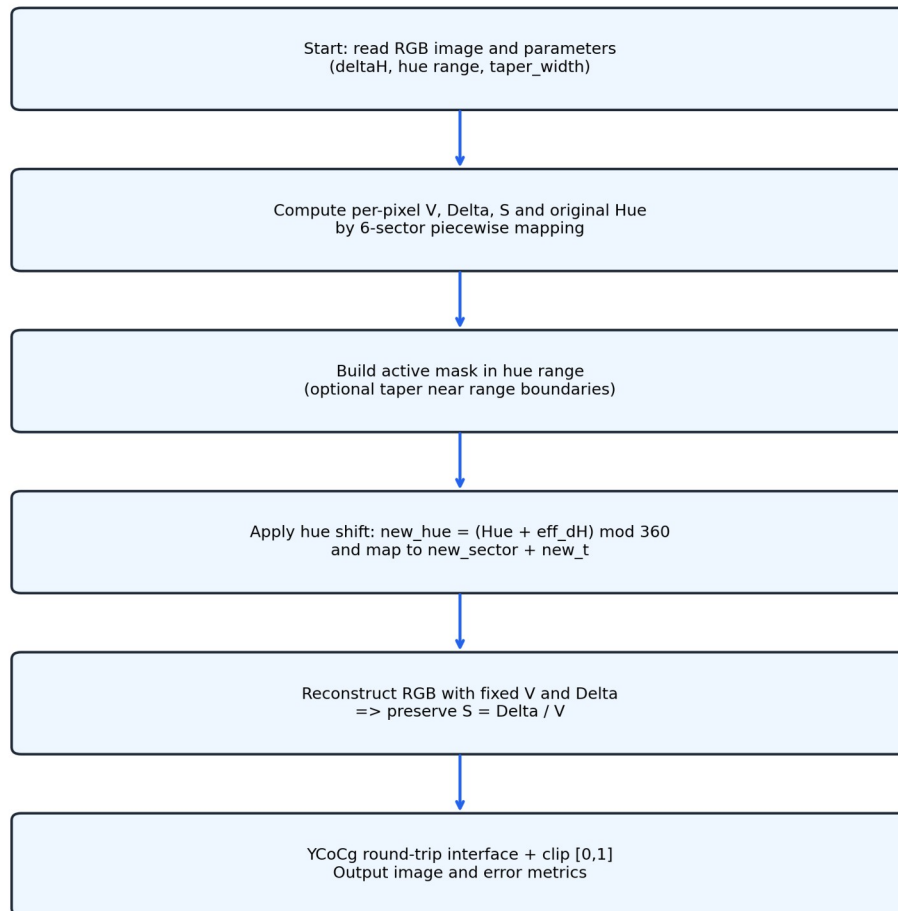
$$\max ErrV = \max |V_{out} - V_{in}|, \quad \max ErrS = \max |S_{out} - S_{in}| \quad (3-14)$$

并可附加 MAE/MaxE(R,G,B) 作为与参考浮点实现的一致性指标。

4. 标准流程图 (简化版)

流程图如下，合并了细枝末节，保留核心处理主线。

FCA.py Flowchart (Simplified)



5. 算法作用后对比图

以下对比图严格使用两个指定文件: soap-bubbles-nature.jpg 与 out_hue_strict_vs.png。

Before vs After (Exact Source Files)



6. 工程建议

- 建议设置 $|\Delta H|$ 的上限阈值，避免过大位移带来非期望颜色迁移。
- 建议保持 $\text{taper_width} \geq \Delta H_{\text{max}}$ ，以进一步抑制范围边界不连续。
- 低饱和度区域可增加 Δ 最小阈值，减少噪声触发。

7. 定点实现所需硬件资源 (参考 FCA_fixedpoint.py)

本节给出“按算法结构推导”的资源模型，用于硬件规划。以下为计算单元需求，不包含片上总线与外设资源。

资源类型	主要用途	典型位宽	并行/复用需求
比较器	计算 V/m 与扇区判定 ($R \geq G$ 等)	8-10 bit	每像素需多路比较; 流水可复用
加法器/减法器	Δ , 通道差分, 重建加减	9-18 bit	主路径必需, 数量与并行像素数线性相关
乘法器	$\Delta * t_{\text{new}}$ 与 taper 缩放	约 $(8 + \text{frac_bits})$ bit	可 1-2 个乘法器时分复用
除法器	$t = \text{diff} / \Delta$ 与 $\text{scale} = d / \text{taper}$	被除数约 $(8 + \text{frac_bits})$ bit	可用迭代除法器复用降低面积
移位器	$\gg \text{frac_bits}$ 与 $\ll$	与数据通路同宽	固定移位, 成本较低

	frac_bits		
逻辑单元	掩码组合与扇区多路选择	1 bit 掩码网络	与像素并行度线性增长
寄存器	流水级缓存与中间量保持	若干 8-18 bit 寄存器组	建议按级插入以满足时序

存储资源建议:

- 1) 行缓存/帧缓存: 若逐像素流式处理, 可不需要全帧缓存, 仅需输入输出行缓冲。
- 2) 中间变量寄存: 需保留 V, m, Delta, hue_hu/new_hue, new_sec, new_t 等。
- 3) 参数寄存: hmin/hmax, deltaH_hu, taper_hu, frac_bits, delta_eps。

面积-性能估算公式 (以每时钟处理 P 个像素计):

比较器数量 $\sim O(P)$, 加减器数量 $\sim O(P)$, 乘法器与除法器数量取决于是否时分复用。

若采用单通道复用除法器, 吞吐降低但面积显著下降; 若全并行, 则吞吐最大但 DSP/逻辑占用更高。